

Kubernetes

Tema 5 - Parte 1

Juan Gutiérrez

Computación en la Nube

Máster en Tecnologías Web, Computación en la Nube y
Aplicaciones móviles

Universitat de València

Outline

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

1 *Introducción*

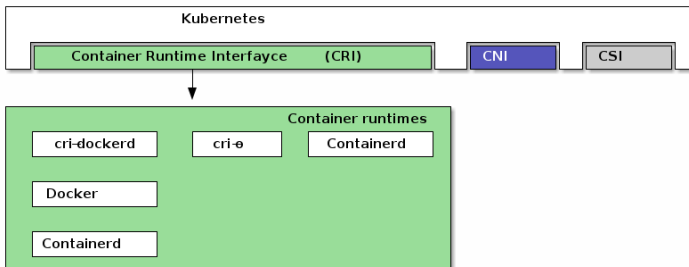
- *Introducción*

Motivación

- Ejecutar contenedores sobre un cluster de máquinas físicas o virtuales (sin preocuparnos de qué máquina tiene que ejecutarlo).
- Descripción declarativa (en fichero YAML) de lo que deseamos ejecutar, k8s (Kubernetes) se encarga de conseguir el estado deseado y de que se cumpla a lo largo del tiempo.
- Posibilidad de usar diferentes *runtime containers* (CRI)
- Posibilidad de usar diferentes gestores de la red (CNI).
- Posibilidad de usar diferentes backends de almacenamiento (CSI).

Motivación

Ejemplo de posibles runtimes para la ejecución de contenedores



Instalación

Máster

- Dependencias (activar módulos del kernel, deshabilitar swap, etc)
- Instalar container runtime (docker + cri-dockerd, cri-o, containerd,...) y configuración
- Instalar kubeadm, kubelet, kubectl
- Iniciar el cluster (`kubeadm init`) con los parámetros adecuados.
- Elegir e instalar el networking (flannel, calico, cilium, weave, etc).
- Si solo hay un nodo con K8s (máster) activarlo para que ejecute pods

En entornos en producción debería haber tres nodos máster, para alta disponibilidad, detrás de un balanceador de carga.

Workers

- Dependencias (activar módulos del kernel, deshabilitar swap, etc)
- Instalar container runtime (docker + cri-dockerd, cri-o, containerd,...) y configuración
- kubeadm, kubelet, kubectl
- Unir el nodo al clúster. Si ejecutamos en el máster `kubeadm token create --print-join-command`, nos da la instrucción a ejecutar en los workers para unirlos al clúster.

Instalación

No es la cosa más complicada del mundo, pero puede llevar unas horas (existen playbooks para Ansible que pueden simplificar la tarea).

Adicionalmente, si queremos integrar k8s con el plano de control de una nube propia, por ejemplo OpenStack, para crear volúmenes en la nube dinámicamente que se puedan vincular a un pod, o crear un balanceador de carga para exponer un servicio al exterior) la cosa se complica un poco más (varios días).

Existen soluciones sencillas que permiten usar K8s en local de una forma rápida: minikube, microk8s, kind, k3s, etc.

<https://blog.palark.com/small-local-kubernetes-comparison/>

Otras alternativas: minishift (openshift, de RedHat, en una máquina virtual).

Proyecto que parece interesante para la instalación de un cluster de kubernetes: <https://docs.k0sproject.io/stable/>

Componentes distribuidos para gestionar k8s

En Kubernetes hay una serie de componentes distribuidos en los nodos que forman el clúster.

Hay dos tipos de componentes:

- **Componentes del plano de control**
- **Componentes de nodo (donde se ejecutan los contenedores)**

Componentes distribuidos en k8s

K8s es declarativo, el usuario define el estado deseado y k8s se encarga de que ese estado se cumpla:

- Se envía al API un objeto (que describe un recurso en formato YAML), este objeto se almacena persistentemente (hasta que el usuario elimina el objeto) y el sistema se encarga de crear y mantener lo que se ha solicitado.

Pero a lo largo de la ejecución es posible que:

- Un contenedor caiga
- El nodo en el que el contenedor se esta ejecutando caiga

Los nodos: se comunican con el API para consultar hacia qué estado deben ir y para informar de su estado.

De este modo en lugar de sobrecargar al máster para que monitorice todos los nodos, se reparte la responsabilidad a cada nodo.

Si el nodo con el API cae, los nodos ejecutan el último estado conocido.

Componentes del plano de control

Son componentes que se instalan en los nodos máster. Se pueden instalar en varios nodos máster para ofrecer alta disponibilidad (necesario si vamos a producción).

kube-apiserver

Es el servicio que expone el API de kubernetes, es decir, se encarga de recibir las peticiones y actualiza el estado del sistema.

etcd

Sirve para mantener el estado del sistema. Es un almacén de datos persistente y distribuido de clave-valor utilizado para almacenar la información. etcd usa el algoritmo de consenso Raft (con elección de líder, máquina de estados y log replicado).

Animación donde se ilustran estos conceptos: <http://thesecretlivesofdata.com/raft/>
Publicación donde se propone Raft:

<https://www.usenix.org/system/files/conference/atc14/atc14-paper-ongaro.pdf>

Componentes del plano de control

kube-scheduler

Es el planificador de k8s: cuando hay que ejecutar una carga de trabajo se encarga de seleccionar un nodo del clúster. Para decidir el nodo se tienen en cuenta diferentes factores: requisitos de recursos, restricciones de hardware/software/políticas, especificación del usuario, etc..

kube-controller-manager

Hay diferentes controladores que se encargan del funcionamiento a lo largo del tiempo de los recursos creados dentro de kubernetes.

Por ejemplo, algunos recursos que ofrece kubernetes permiten especificar el **número de réplicas** a ejecutar. Puede darse el caso de que se produzca algún error y el nodo que ejecuta una réplica falle. El controller de replicación se encarga de comprobar el estado real de un recurso y si no coincide con la especificación solicitada, automáticamente toma las acciones necesarias para que se cumpla.

Componentes del plano de control

cloud-controller-manager

Se encargan de comunicarse con el plano de control de la infraestructura de computación en la nube para (entre otras tareas):

- Gestionar balanceadores de carga
- Gestionar volúmenes
- Detectar y actuar cuando un nodo deja de funcionar

Por ejemplo: <https://github.com/kubernetes/cloud-provider-openstack>

Componentes en los nodos worker

Se instalan en nodos worker (donde se van a ejecutar los recursos solicitados por los usuarios) y se encargan de mantener los pods en funcionamiento y configurar el networking.

En estos nodos debe haber un runtime para la ejecución de contenedores. K8s soporta: Docker, containerd, cri-o, rkt, o cualquier interfaz de ejecución de contenedores que cumpla con Kubernetes CRI.

kubelet

Se asegura de que los contenedores se ejecutan en un pod. A partir de la especificación para el pod proporcionada por el usuario, se encarga de que esa especificación se cumpla.

kube-proxy

Se encarga de configurar la red en el nodo y hacer el reenvío para que se puedan comunicar componentes que están en diferentes nodos.

*Elementos adicionales**DNS*

Un clúster k8s debe tener un DNS interno para permitir descubrir servicios por su nombre. Los contenedores que se ejecutan dentro de kubernetes, incluyen este DNS para resolver la IP a partir del nombre.

Dashboard (panel de control web)

Aplicación web para la administración y resolución de problemas.

Servidor de métricas

Obtención y almacenamiento centralizado de métricas a lo largo del tiempo (series temporales). Las métricas se pueden obtener de los nodos, de los contenedores, etc. Se suele usar **Prometheus** (para la recuperación de las métricas) y **Grafana** (para su visualización).

Registros (logs) del clúster

Obtención y almacenamiento centralizado de los logs.

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

- 2 [API](#)
- [API](#)

API

Kubernetes expone un API que atiende a peticiones HTTPS.

Se ofrecen diferentes APIs REST.

Por ejemplo, bajo `/api/v1/` se encuentran los elementos principales (*core*):

- `/api/v1/pods`
- `/api/v1/namespaces`
- `/api/v1/nodes`
- `/api/v1/configmaps`
- etc.

Por ejemplo si realizamos una petición a `/api/v1/pods` obtendremos un listado de los recursos del tipo pod.

API

Bajo `/apis/GRUPO/VERSIÓN` se encuentran:

- `/apis/apps/v1/deployments`
- `/apis/apps/v1/statefulsets`
- `/apis/networking.k8s.io/v1/networkpolicies`
- etc.

Por ejemplo si realizamos una petición a `/apis/apps/v1/deployments` obtendremos un listado de los recursos del tipo `deployment`.

API

Si creamos un nuevo objeto (por ejemplo un deployment), el API le asocia una URL a la que se pueden hacer peticiones REST para:

- Modificar el recurso (pasando en el cuerpo la nueva especificación en formato JSON).
- Consultar el estado del recurso (petición GET)
- Borrar el recurso (petición DELETE).

Analogía: en el ejemplo de la caché del tema 1, una vez creado un par clave/valor lo podíamos consultar o borrar.

Versionado de la API

- Si el API tiene en su nombre `alpha` (por ejemplo `v1alpha1`) significa que está en desarrollo y puede contener errores. No se garantiza su soporte a largo plazo. Se debería usar solo en entornos de prueba.
- El nombre de versión `beta` (por ejemplo `v2beta3`) implica que el código ha sido probado, pero los detalles podrían variar en una versión posterior. No se recomienda su uso para aplicaciones críticas.
- En el nivel estable el nombre de la versión es `vX`, donde `X` es un entero, se asegura que las características se mantendrán en las siguientes versiones de kubernetes.

API

Podemos hacer peticiones con curl a la API.

La forma más sencilla es usar `kubectl proxy` que redirige las peticiones locales realizadas al puerto 8001 a kube-apiserver (el servidor del API):

```
# Por defecto escucha en el puerto 8001
```

```
kubectl proxy &
```

```
# Todos los pods en ejecución:
```

```
curl localhost:8001/api/v1/pods
```

```
# Todos los namespaces
```

```
curl localhost:8001/api/v1/namespaces
```

kubectl para realizar peticiones al API de kubernetes

La forma de interactuar con kubernetes **no** es mediante curl.

Desde el CLI usaremos kubectl que internamente realiza peticiones HTTPS a la API.

Esta aplicación, por defecto, lee un fichero de configuración, que se encuentra en el directorio /home/user/.kube/config, y que contiene la información para acceder a la API (ip, puerto, certificado, usuario, etc).

Por lo tanto, necesitamos obtener ese fichero de kubernetes para acceder desde la máquina física:

```
mkdir ~/.kube  
scp k8s@192.168.56.3:~/.kube/config ~/.kube
```

Comprobación:

```
kubectl get-nodes
```

kubectl para realizar peticiones al API De kubernetes

En el fichero config aparece la IP y el puerto donde escucha el servidor del API, certificados para que el servidor valide el acceso, el usuario, etc:

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: XXXXXX
    server: https://192.168.56.3:6443
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
    name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: XXXXXX
    client-key-data: XXXXXX
```

Ojo, en este caso como vemos, accedemos con un usuario con permisos de administración.

En un escenario real habría que crear usuarios con permisos restringidos.

kubectl para realizar peticiones al API de kubernetes

Veremos ejemplos del uso de `kubectl` a lo largo de la presentación.

La forma de estas instrucciones es:

```
kubectl action resource
```

Donde indicamos que realice una **acción** (crear, describir, borrar, etc) sobre el **recurso** indicado (nodo, pod, deployment, etc).

Podemos pasar `--help` para obtener información sobre los parámetros específicos. Por ejemplo:

```
kubectl get nodes --help
```

kubectl para realizar peticiones al API de kubernetes

Si no sabemos a qué API (grupo y versión) pertenece un recurso podemos ejecutar `kubectl api-resources` y filtrar por el tipo de recurso:

```
kubectl api-resources | grep pod
```

NAME	SHORT NAME	API VERSION	NAMESPACED	KIND
pods	po	v1	true	Pod

```
kubectl api-resources | grep deployment
```

NAME	SHORT NAME	API VERSION	NAMESPACED	KIND
deployments	deploy	apps/v1	true	Deployment

```
kubectl api-resources | grep hpa
```

NAME	SHORT NAME	API VERSION	NAMESPACED	KIND
horizontalpodautoscalers	hpa	autoscaling/v2	true	
HorizontalPodAutoscaler				

Algunos ejemplos de uso de kubectl

```
kubectl cluster-info
kubectl get nodes
kubectl get pods

# Para obtener información de un recurso del tipo pod
kubectl get pod NOMBRE
kubectl get pod NOMBRE -o wide
# Salida en formato yaml
kubectl get pod NOMBRE -o yaml
# Salida en formato json
kubectl get pod NOMBRE -o json

# Dato concreto (en este caso el UID del pod)
kubectl get pod NOMBRE -o jsonpath="{.metadata.uid}"
kubectl get pod NOMBRE -o json | jq .metadata.uid

kubectl describe pod NOMBRE

kubectl cp NOMBRE-POD:RUTA-EN-POD RUTA-LOCAL

# Para crear o modificar un nuevo recurso definido en el fichero yaml
kubectl apply -f fichero.yaml
```

Especificación de los recursos

Para crear un recurso en k8s se realiza una petición al API enviando una especificación declarativa que proporcionamos en un fichero yaml.

Los campos requeridos en el fichero yaml son:

- `apiVersion` qué versión del API se usa para crear el objeto
- `kind` el tipo de objeto a crear
- `metadata` nombre del objeto, espacio de nombres en el que se debe crear, etiquetas, anotaciones
- `spec` con la especificación del objeto y cuyo contenido dependerá del tipo de objeto (pod, deployment, job, service, etc) que estemos creando.

En el siguiente enlace se puede consultar la documentación para el campo `spec` de los diferentes objetos:

<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31>

Obtención de los ejemplos de código

```
mkdir ~/Tema2
cd ~/Tema2
# git clone https://estudianteTWCAM@routerdi1315.uv.es:10000/curso-k8s/ejempl
↪ os-k8s-partel.git
# Si tenemos la variable de entorno REPO:
git clone $REPO/ejemplos-k8s-partel.git
```

Ejemplo de especificación de un recurso

Fichero: api/nginx.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-server
  namespace: ns-api
spec:
  containers:
    - name: nginx
      image: nginx:1.29.5-alpine
      ports:
        - containerPort: 80
```

Para crear este objeto en kubernetes podemos usar kubectl:

```
kubectl apply -f nginx.yaml
```

Especificación de un recurso

Cuando estamos en desarrollo (esto **no se hace así en producción**) podemos hacer una redirección de puertos desde nuestra máquina hasta el puerto de la aplicación que se ejecuta en el pod:

```
kubect1 port-forward nginx-server --address 0.0.0.0 9000:80 -n ns-api
```

En esta instrucción estamos especificando:

- ❶ Que se debe hacer una redirección para enviar las peticiones al pod `nginx-server`
- ❷ Que el puerto local es 9000
- ❸ Que el puerto de la aplicación dentro del pod es 80
- ❹ Que debe escuchar en todas las interfaces (por defecto solo lo hace en la interfaz de *loopback*)

Si abrimos un navegador y ponemos la URL <http://localhost:9000> nos debería contestar el servicio que hemos desplegado en el pod.

Especificación de un recurso

Para saber qué podemos usar en la especificación de cada recurso hay que consultar la documentación.

Por ejemplo, los campos que podemos usar en la especificación de un pod son:

<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#podspec-v1-core>

Los campos que podemos usar para cada container del pod:

<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#container-v1-core>

Cuando se pide a kubernetes la descripción de un objeto que ha sido creado, añade un campo `status` que contiene el estado actual del recurso.

```
kubectl get pod nginx-server -o yaml -n ns-api
```

Tarea: creación de un namespace de forma declarativa

- 1 Busca en la documentación del API información sobre el recurso del tipo Namespace. También puedes usar `kubectl explain namespace`.
- 2 Crea un fichero `yaml` donde se especifique un Namespace con el nombre `ns-prueba`.
- 3 Usa `kubectl` para crear el namespace.
- 4 Usa `kubectl` borrar el namespace creado.

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

- 3 *Organización de recursos en Namespaces*
 - Namespaces

Introducción

Los espacios de nombres permiten evitar nombres repetidos (no podemos crear dos recursos con el mismo nombre) y representan un *scope* para los nombres de los objetos.

Un posible uso de los espacios de nombres es para agrupar los objetos por equipo de desarrollo, departamento o responsabilidad.

Si no especificamos un *namespace* al crear un recurso, Kubernetes asume que está en el namespace `default`.

Introducción

Durante el proceso de instalación de kubernetes se crean diferentes namespaces, entre los que se encuentran:

- `default` espacio de nombres por defecto
- `kube-system` espacio de nombres que contiene recursos encargados de la infraestructura.
- `kube-node-lease` contiene objetos que permiten a kubelet enviar *heartbeats* hacia el plano de control. De este modo el plano de control puede detectar fallos en los nodos.

Trabajar con namespaces

```
# Lista los namespaces  
kubectl get namespaces
```

```
# Crea un namespace con el nombre indicado  
kubectl create namespace <nombre-namespace>
```

O bien usando YAML:

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: nombre-namespace
```

Y enviado la petición con el contenido del fichero YAML a Kubernetes:

```
kubectl apply -f namespace.yaml
```

Trabajar con namespaces

Obtener los recursos del tipo pod (sustituir pod por el recurso que se quiere consultar) que hay en un namespace:

```
kubectl get pod --namespace <nombre-namespace>  
  
kubectl get pod -n <nombre-namespace>
```

Eliminar un namespace (esto **borra todos los recursos** que se han creado en el namespace):

```
kubectl delete namespace <nombre-namespace>
```

Listar todos los recursos del tipo pod en todos los namespaces:

```
kubectl get pods --all-namespaces
```

Indicar que un recurso pertenece a un namespace

Al crear un recurso, el namespace al que pertenece se especifica como un metadato en el fichero YAML (campo `metadata.namespace`):

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: front
# Resto de datos
```

Cuando lo enviemos al API:

```
kubectl apply -f nginx.yaml
```

El recurso `nginx` se crea en el namespace `front`.

Instrucción para establecer el namespace de trabajo

Si vamos a ejecutar muchas instrucciones en un namespace, puede ser conveniente indicar que vamos a trabajar con ese namespace (en lugar de en default):

```
kubectl config set-context --current --namespace=front
```

De este modo al ejecutar `kubectl` se trabaja en ese namespace:

```
kubectl get pod
```

Se estarán mostrando los recursos que están contenidos en el namespace `front`.

Tarea sobre namespaces I

- 1 Lista todos los namespaces
- 2 Clona el repositorio:

```
mkdir -p ~/Tema2 && cd ~/Tema2
git clone https://estudianteTWCAM@routerdi1315.uv.es:10000/curso-k8s/tarea-namespaces.git
↪ area-namespaces.git
# git clone $REPO/tarea-namespaces.git
cd tarea-namespaces
```

- 3 Ejecuta:

```
kubectl apply -f .
```

- 4 La instrucción anterior crea dos namespaces: ns-app1 y ns-app2. Además crea un pod en cada namespace.
- 5 Comprueba que si solicitamos los pods y no especificamos el namespace no se muestra nada.
- 6 Muestra de nuevo todos los namespaces
- 7 Muestra los pods creados en el namespace ns-app1

Tarea sobre namespaces II

- 8 Muestra los pods creados en el namespace ns-app2
- 9 Borra el namespace ns-app1 y comprueba si existen los pods de ese namespace.
- 10 Borra el namespace ns-app2 y comprueba si existen los pods de ese namespace.

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

- 4 *Etiquetas y selectores*
 - Etiquetas y selectores

Etiquetas

Con las arquitecturas basadas en microservicios, es fácil acabar con cientos de recursos en ejecución dentro del cluster de Kubernetes. Sin un mecanismo que permita organizarlos tendríamos un buen lío.

Podemos añadir metadatos en los recursos en forma de etiquetas que se pueden usar posteriormente para: consultar, filtrar y ordenar recursos.

Las etiquetas son pares del tipo clave/valor (donde la longitud del valor está limitada a 63 caracteres alfanuméricos y separadores).

Declaración de etiquetas

En el siguiente ejemplo se asignan dos etiquetas a un recurso:

Fichero: labels-annotations/nginx-etiquetado.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: ns-labels
  labels:
    env: dev
    tier: front
spec:
  containers:
  - image: nginx:1.29.5-alpine
    name: nginx
```

```
kubectl apply -f nginx-etiquetado.yaml
```

También podemos añadir etiquetas a un recurso que ya esté desplegado:

```
kubectl label pod nginx team=team1 -n ns-labels
```

Si un recurso ya tiene una etiqueta y queremos cambiar su valor hay que pasar la opción `--overwrite`.

Inspeccionar las etiquetas

Podemos pedir que la información de un recurso se muestre en formato YAML y la podemos procesar con `yq` (YAML query):

```
kubectl get pod nginx -n ns-labels -o yaml | yq '.metadata.labels'
```

```
team: team1
env: dev
tier: front
```

Si queremos ver todas las etiquetas que tienen los recursos de un determinado tipo:

```
kubectl get pods --show-labels -n ns-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
nginx	1/1	Running	0	11s	team=team1,env=dev,tier=front

Ejemplos de uso de las etiquetas para seleccionar recursos I

Al usar `kubectl` podemos usar las etiquetas para seleccionar recursos. Ejemplos de posibles criterios:

- 1 El valor de la etiqueta tiene que coincidir con el proporcionado:

```
kubectl get pod -l team=team1 -n ns-labels  
# Eso es equivalente  
# kubectl get pod --selector app=aula-virtual -n ns-labels
```

- 2 El valor de la etiqueta tiene que coincidir con alguno de los elementos de un conjunto:

```
kubectl get pod -l 'team in (team1,team2)' -n ns-labels
```

- 3 El valor de la etiqueta no debe coincidir con el proporcionado:

```
kubectl get pod -l "tier != backend" -n ns-labels
```

- 4 Combinación de las anteriores (en este caso no hay ningún recurso que lo cumpla):

```
kubectl get pod -l "tier=front,env != dev" -n ns-labels
```

Uso de selectores en ficheros YAML

Al declarar algunos recursos es necesario indicar a qué elementos se aplica lo que estamos creando.

Por ejemplo, un Deployment permite gestionar pods (por ejemplo manteniendo un número determinado de réplicas en ejecución). Por tanto, al crear un Deployment hay que especificar cuales son los pods que gestiona.

En este caso, se puede ver que se aplica a los pods que tengan la etiqueta app con el valor myapp-pod:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  selector:
    matchLabels:
      app: myapp-pod
# Resto del fichero donde se define el pod (que tendrá la etiqueta app con ese
↪ valor)
```

Etiquetas y selectores para los nodos

Si etiquetamos los nodos, podemos usar estas etiquetas para que el planificador seleccione nodos donde ejecutar el recurso.

Para ver las etiquetas que tiene un nodo:

```
kubectl get node k8sVM -o=yaml | yq -M '.metadata.labels'
```

```
beta.kubernetes.io/arch: amd64
beta.kubernetes.io/os: linux
kubernetes.io/arch: amd64
kubernetes.io/hostname: siuv-k8s
kubernetes.io/os: linux
node-role.kubernetes.io/control-plane: ""
node.kubernetes.io/exclude-from-external-load-balancers: ""
```

Etiquetas y selectores para los nodos

Supongamos que este nodo tiene una GPU y queremos añadir una etiqueta que represente este hecho:

```
kubectl label node k8sVM gpu=true
```

```
kubectl get node k8sVM -o=yaml | yq -M '.metadata.labels'
```

```
gpu: "true"  
# Resto de etiquetas
```

Etiquetas y selectores para los nodos

Podemos usar esta etiqueta para especificar que el recurso se debe ejecutar en un nodo que tenga la etiqueta que hemos añadido usando `nodeSelector` en la especificación:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp
spec:
  nodeSelector:
    gpu: true
  containers:
  - name: myapp
    # Resto de datos
```

Ojo, si no tenemos ningún nodo con esa etiqueta entonces no se podrá ejecutar el recurso.

Tarea sobre etiquetas

- ➊ Añadir la etiqueta `cpd` con valor `burjassot` al nodo `k8sVM`.
- ➋ Mostrar las etiquetas del nodo para comprobar que se ha etiquetado.
- ➌ Cambiar el valor de la etiqueta `cpd` al valor `tarongers` en el nodo `k8sVM`.
- ➍ Mostrar las etiquetas del nodo para comprobar que se ha modificado el valor de la etiqueta.
- ➎ Buscar en la documentación (`kubectl label --help`) la forma de eliminar la etiqueta añadida.
- ➏ Eliminar la etiqueta y comprobar que ya no está.

Anotaciones

Las anotaciones sirven para añadir metadatos, pero a diferencia de las etiquetas, no se utilizan para seleccionar objetos. Añaden información adicional que puede ser útil para los usuarios o para que otros recursos desplegados en el cluster realicen alguna acción.

Cada anotación también es un par clave valor, pero a diferencia de las etiquetas, el valor en una anotación puede contener espacios y otros caracteres no admitidos en las etiquetas (si es así se debe poner entre comillas).

Listar las anotaciones de un objeto

```
kubect1 get node k8sVM -o=yaml | yq -M .metadata.annotations
```

```
flannel.alpha.coreos.com/backend-data: '{"VNI":1,"VtepMAC":"6a:e0:6f:bd:2c:78"}'  
flannel.alpha.coreos.com/backend-type: vxlan  
flannel.alpha.coreos.com/kube-subnet-manager: "true"  
flannel.alpha.coreos.com/public-ip: 10.0.2.15  
kubeadm.alpha.kubernetes.io/cni-socket: unix:///var/run/cni-dockerd.sock  
node.alpha.kubernetes.io/ttl: "0"  
volumes.kubernetes.io/controller-managed-attach-detach: "true"
```

Anotaciones en el fichero YAML

Las anotaciones se especifican en `metadata.annotations`:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp
  label:
    tier: front
  annotations:
    commit: 0f76ff52b
    image-builder: "asperjuni12@uv.es"
    rama: "feature/add-menu"
spec:
  containers:
    - name: myapp
# Resto de datos
```

Añadir una anotación a un recurso en ejecución

```
kubectl annotate pod myapp developer="María"
```

Podemos combinarlo con la selección por etiquetas para añadir una anotación a todos los recursos (en este caso pods) que tengan la etiqueta tier con el valor front:

```
kubectl annotate pod -l tier=front developer="María"
```

Tarea sobre anotaciones

- 1 Añadir las anotaciones `maintainer` con valor `Vicente` y `creation-date` con valor `2025` al nodo `k8sVM`.
- 2 Mostrar las anotaciones del nodo para comprobar que se han añadido las anotaciones.
- 3 Cambiar el valor de la anotación `maintainer` al valor `Ana`
- 4 Comprobar que se ha modificado el valor.
- 5 Buscar en la documentación (`kubectl annotate --help`) la forma de eliminar las anotaciones añadidas.
- 6 Eliminar las anotaciones y comprobar que ya no están.

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

5 *Ejecución de cargas de trabajo*

- Pod
- Deployment
- DaemonSet
- Job y CronJob

Introducción

- El pod es la unidad mínima de despliegue de aplicaciones en Kubernetes.
- Un pod puede consistir de un solo contenedor o puede contener varios contenedores. Cada contenedor ejecuta un proceso o servicio.
- Los contenedores de aplicación que forman un pod se inician en paralelo.
- Cada pod tiene una única dirección IP que es compartida por todos los contenedores que lo componen.
- Los contenedores de un pod se pueden comunicar entre ellos usando IPC, la interfaz loopback o un sistema de ficheros compartido.

Introducción

Por tanto, todos los contenedores de un pod:

- Comparten el mismo nombre de host e interfaces de red
- Tienen la misma dirección IP y espacio de puertos (por tanto, cada contenedor de un pod debe usar puertos diferentes al resto de contenedores).
- Pueden compartir información entre ellos mediante ficheros.

Además, El pod es la unidad básica de escalado (si un pod se escala, lo hacen todos los contenedores que forman el pod).

Introducción

- Los pods que contienen un único contenedor son los más habituales. El contenedor ofrece un determinado servicio (Nginx, MySQL, Microservicio, etc).
- La necesidad de ofrecer más de un contenedor viene dada por:
 - ▶ Hay contenedores de inicialización (**initContainers**) que se ejecutan secuencialmente antes que los contenedores de aplicación. Deben ejecutar una aplicación que finalice relativamente pronto. Si un contenedor de inicialización falla, falla la ejecución del pod.
 - ▶ Hay contenedores que ofrecen algún servicio adicional (**sidecar**). Por ejemplo, pueden actuar como proxy (interceptan la petición que va al contenedor de aplicación y realizan alguna función). Hay un proxy ampliamente usado: <https://www.envoyproxy.io/>.

Introducción

Normalmente, no ejecutaremos un pod directamente sino que usaremos otros recursos que gestionan y controlan a los pods:

- **Deployment** (para aplicaciones sin estado que se están ejecutando de forma indefinida, donde se puede especificar el número de réplicas, variar el número de réplicas a lo largo del tiempo, realizar actualizaciones, volver atrás si hay problemas, etc)
- **DaemonSet** (que permite ejecutar un pod en cada nodo del cluster)
- **Job** (que permite encapsular un trabajo que realiza alguna tarea y finaliza).
- **CronJob** (que permite programar la ejecución de un job)
- **StatefulSet** (para almacenar el estado en varias réplicas, donde cada réplica tiene un rol diferenciado).

Ciclo de vida de un pod

Al solicitar que se ejecute un pod, el nodo al que se le asigne la ejecución debe:

- Descargar la imagen (o imágenes si se trata de un pod con múltiples contenedores). Por esto es importante que las imágenes sean pequeñas.
- Preparar el entorno de ejecución para los procesos y el networking
- Ejecutar los contenedores que se han especificado

Ciclo de vida de un pod

<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>

Tras ejecutar un pod podemos ver en qué fase está su ejecución:

- **Pending:** el pod ha sido aceptado por Kubernetes pero alguno de sus contenedores aún no ha sido creado.
- **Running:** el pod se ha asociado a un nodo y todos los contenedores se han creado. Al menos un contenedor está en ejecución o está en proceso de inicio o reinicio.
- **Succeeded:** todos los contenedores del pod han finalizado, y no serán reiniciados.
- **Failed :** todos los contenedores del pod han finalizado y al menos un contenedor ha terminado con un estado diferente de cero o lo ha detenido el sistema.
- **Unknown:** el estado del pod no se ha podido determinar, habitualmente por no poder comunicarse con el nodo que lo está ejecutando.

```
kubectl get pod <pod-name>
```

Estado de los contenedores en un pod

Además, los contenedores que forman un pod pasan por diferentes fases:

- **Waiting:** se están ejecutando las operaciones necesarias para que el contenedor se inicie.
- **Running:** el contenedor se está ejecutando
- **Terminated:** el contenedor no se está ejecutando, bien porque ha fallado su ejecución bien porque el proceso que se ejecutaba en el contenedor ya ha finalizado.

Podemos ver el estado de los contenedores de un pod con la siguientes instrucción:

```
kubectl describe pod <pod-name>
```

¿Qué contiene un fichero YAML de un Pod?

La especificación de un pod en un fichero YAML consta de las siguientes partes:

- `apiVersion` para este recurso `v1`
- `kind` en este caso `Pod`
- `metadata` que incluye el nombre, namespace, etiquetas e información adicional.
- `spec` especificación del pod: un array con los `initContainers`, un array con los contenedores, un array con los volúmenes, etc.

Especificando los recursos para los contenedores en un pod

Al especificar un contenedor podemos indicar:

- `spec.containers.resources.limits`: el consumo máximo permitido de recursos (cpu y memoria)
- `spec.containers.resources.requests`: el consumo solicitado (CPU y memoria)

La CPU se indica en millicores (200m) o en cores (0.2) y para la memoria especificamos las unidades (Ki, Mi, Gi).

Ejemplo

En este ejemplo indicamos que el contenedor solicita 128 Mi y 500 milicores, de este modo el planificador tiene más información sobre dónde colocar este pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.29.5-alpine
    resources:
      requests:
        memory: "128Mi"
        cpu: "500m"
    ports:
      - containerPort: 80
```

Si ningún nodo tiene los recursos necesarios para ejecutar el pod, entonces permanece en estado *Pending*.

Paso de variables de entorno

A cada contenedor del pod le podemos pasar un array de variables de entorno mediante `spec.containers[].env`.

Esto se usa para configurar la aplicación (sin tener que pasar por el ciclo de compilar, construir la imagen del contenedor y subir la imagen del contenedor a un registro).

`env` espera un array donde cada entrada es un par donde indicamos el nombre y el valor.

Ejemplo

Fichero: pods/ejemplo2-pod-env.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-env
  namespace: ns-pods
spec:
  containers:
    - name: nginx
      image: nginx:1.29.5-alpine
      env:
        - name: TOKEN
          value: "36643f38-a891-11ee-9de1-a7fe5f301ccb"
        - name: URL_PATH
          value: "/cache"
      ports:
        - containerPort: 80
```

```
kubectl apply -f ejemplo2-pod-env.yaml
```

Ejemplo

Podemos indicar a `kubectl` que se bloquee hasta que el pod esté listo: Ejecutamos el proceso `env` dentro del pod para listar las variables de entorno:

```
kubectl exec nginx-env -n ns-pods -- env
```

```
TOKEN=36643f38-a891-11ee-9de1-a7fe5f301ccb  
URL_PATH=/cache  
# Resto de las variables de entorno
```

Si algo va mal: obtención de los logs y una shell en un pod

Estas instrucciones son útiles cuando hay algún problema:

Obtención de los logs:

```
kubectl logs <pod-name>
```

Obtener una shell:

```
kubectl exec -it <pod-name> -- /bin/sh  
# o si tiene Bash  
kubectl exec -it <pod-name> -- /bin/bash
```

Ejecutar un proceso:

```
# Listar las variables de entorno  
kubectl exec -it <pod-name> -- env  
# Listar contenido de un directorio  
kubectl exec -it <pod-name> -- ls /app
```

Especificar el proceso a ejecutar al iniciar un contenedor del pod

Tal y como vimos en el tema sobre Docker, al crear una imagen podíamos especificar el proceso a ejecutar al iniciar el contenedor (bien con `CMD` bien con `ENTRYPOINT`).

En kubernetes podemos indicar con `spec.containers[].command` el proceso a ejecutar y con `spec.containers[].args` los argumentos que se pasan al proceso.

- Si no pasamos `command` se ejecuta el que esté establecido en la imagen con `CMD` o con `ENTRYPOINT`.
- Si pasamos `command` se ejecuta el indicado, ignorando el establecido con `CMD` o con `ENTRYPOINT`.
- Si pasamos `args` se ejecutará el establecido en `CMD` o en `ENTRYPOINT` pero con los argumentos proporcionados.

Ejemplo

Usamos una image busybox y especificamos el proceso a ejecutar:

Fichero: pods/ejemplo3-pods.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: busybox
  namespace: ns-pods
spec:
  containers:
  - name: busybox
    image: busybox:1.37.0
    command: ["/bin/sh"]
    args: ["-c", "for i in $(seq 1 30); do echo $i; sleep 1; done"]
```

```
kubectl apply -f ejemplo3-pods.yaml
```

Mostrar los logs tal y como se van generando (con la opción `-f` o `--follow`):

```
kubectl logs -f busybox -n ns-pods
```

Eliminando Pods

Cuando se solicita eliminar un pod, kubernetes intenta completar las peticiones activas al pod.

Una petición de borrado puede tardar, habitualmente, entre 5 y 30 segundos.

```
# Borra el pod cuyo nombre se indica
kubectl delete pod <nombre-pod>

# Borra el pod que tiene la etiqueta indicada
kubectl delete pod -l label=value

# Borra todos los pods del namespace indicado
kubectl delete namespace <nombre-namespace>
```

Si tenemos el fichero YAML con la especificación del pod también podemos borrarlo del siguiente modo:

```
kubectl delete -f fichero-pod.yaml
```

restartPolicy

Informa a kubernetes sobre qué hacer con el pod cuando la aplicación finalice. En `spec.restartPolicy` podemos poner tres posible valores:

Always (valor por defecto)

Los contenedores del pod se reiniciarán si las aplicaciones que se ejecutan en ellos finalizan. Independientemente de si se han completado con éxito o no. Esta opción es apropiada cuando en los contenedores ejecutamos un servicio que se debe ejecutar continuamente.

OnFailure

Reiniciará los contenedores solo si los procesos que ejecutan finalizan con error (con un valor diferente de 0). Esta opción es apropiada para contenedores que necesitamos que se ejecuten hasta que finalicen correctamente y entonces ya no se reiniciarán.

Never

No se reiniciarán los contenedores, independientemente de cómo hayan finalizado.

Ejemplo 1

Ejemplo de un pod con un contenedor que ejecuta un servicio de forma continua:

Fichero: pods/ejemplo-pod-nginx.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: ns-pods
spec:
  containers:
    - name: nginx
      image: nginx:1.29.5-alpine
      ports:
        - containerPort: 80
```

```
kubectl apply -f ejemplo-pod-nginx.yaml
```

Ejemplo 2 /

Ejemplo de un pod con un contenedor que ejecuta una aplicación y finaliza (sin código de error):

Fichero: pods/ejemplo-pod-sleep.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: sleep
  namespace: ns-pods
spec:
  containers:
  - name: sleep
    image: busybox:1.37.0
    command: ["sleep", "5"]
```

```
kubectl apply -f ejemplo-pod-sleep.yaml
```

¿Qué ocurre con este pod?

Ejemplo 2 II

```
kubectl get pods -n ns-pods
```

NAME	READY	STATUS	RESTARTS	AGE
sleep	0/1	CrashLoopBackOff	6 (2m20s ago)	8m45s

Ha reiniciado el contenedor varias veces, hasta el máximo configurado (6 reinicios), ya que hemos usado el valor por defecto, Always, de restartPolicy.

```
kubectl delete pod sleep -n ns-pods
```

Ejemplo 3 /

Ejemplo de un pod con un contenedor que ejecuta una aplicación que tiene una duración determinada (y finaliza sin error). En este ejemplo especificamos `restartPolicy` con el valor `OnFailure`:

Fichero: pods/ejemplo-pod-sleep-reinicio-si-fallo.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: sleep-onfailure
  namespace: ns-pods
spec:
  restartPolicy: OnFailure
  containers:
  - name: sleep
    image: busybox:1.37.0
    command: ["sleep", "5"]
```

```
kubectl apply -f ejemplo-pod-sleep-reinicio-si-fallo.yaml
```

Ejemplo 3 II

¿Qué ocurre con este pod? Que como el proceso ha finalizado correctamente, no lo reinicia.

```
kubectl get pod sleep-onfailure -n ns-pods
```

NAME	READY	STATUS	RESTARTS	AGE
sleep-onfailure	0/1	Completed	0	26s

```
kubectl delete pod sleep-onfailure -n ns-pods
```

Ejemplo 4 I

Ejemplo de un pod con un contenedor que ejecuta una aplicación y da error si no existe un fichero `f.txt`:

Fichero: `pods/ejemplo-pod-sleep-fallo.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: sleep-fallo
  namespace: ns-pods
spec:
  restartPolicy: OnFailure
  containers:
  - name: sleep-fallo
    image: busybox:1.37.0
    command:
    - "/bin/sh"
    - "-c"
    - "sleep 30; if [ -f f.txt ]; then echo OK;else ffmpeg; fi"
```

Ejemplo 4 II

El error se produce ya que `ffmpeg` no está instalado en la aplicación.
¿Qué ocurre con el contenedor?

```
kubectl apply -f ejemplo-pod-sleep-fallo.yaml
```

Que mientras no exista un fichero llamado `f.txt` finalizará con un error.
Podemos dejar varios intentos de reinicio, observando cada segundo lo que ocurre:

```
watch -n 1 kubectl get pod sleep-fallo -n ns-pods
```

Y mientras esté en ejecución podemos crear el fichero dentro del contenedor:

```
kubectl exec sleep-fallo -n ns-pods -- touch f.txt
```

Con eso el proceso finalizará correctamente y ya no se reiniciará.

Ejemplo 4 III

```
kubectl delete pod -n ns-pods sleep-fallo
```

imagePullPolicy

Podemos especificar qué debe hacer con la imagen de un contenedor:

IfNotPresent

La imagen se descarga si no está presente localmente en el nodo

Always

Cada vez que se ejecuta el contenedor, kubelet se comunica con el registro de imágenes para comprobar si la imagen ha cambiado.

Never

No la descarga, si está inicia el contenedor pero si no está fallará.

imagePullPolicy

No se recomienda el uso de la etiqueta `latest` en producción ya que no sabemos qué versión de la imagen se está ejecutando y es más difícil volver atrás en caso de fallo.

Se puede especificar una versión concreta y/o un digest.

Por ejemplo:

```
docker image inspect nginx:1.29.5-alpine | jq '.[0].Id'
```

```
sha256:91ca84b4f57794f97f70443afccff26aed771e36bc48bad1e26c2ce66124ea66
```

Esto es más seguro ya que puede ser que hayan cambiado la aplicación en la imagen manteniendo la etiqueta y podríamos tener varias versiones de la aplicación en ejecución.

Planificación de los pods en los nodos

Hay situaciones en las que queremos mayor control sobre donde queremos que se ejecuten los recursos.

Podemos usar dos mecanismos:

Taints y tolerations (sobre nodos)

Taint es un mecanismo que permite marcar nodos de tal forma que solo aquellos recursos que toleren esta marca se puede ejecutar en esos nodos. Una posible analogía sería esta: infectamos un nodo con un virus de tal modo que los pods no se ejecutarán en ese nodo, a no ser que el pod tolere o esté inmunizado contra ese virus.

Affinity/anti-affinity (sobre nodos o pods)

Podemos hacer que un pod busque nodos con determinadas etiquetas o que los rechace. También podemos hacer que lo ejecute en el mismo nodo que ya esté ejecutando un pod con una etiqueta o que lo ejecute en uno diferente.

Taints y tolerations

<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>

Podemos marcar nodos con la siguiente sintaxis:

```
kubectl taint node NODE ETIQUETA:VALOR:EFECTO
```

donde:

- NODE: es el nodo a etiquetar (o --all=true si lo aplicamos a todos).
- ETIQUETA:VALOR es un par clave/valor (por ejemplo CPU:slow).
- EFECTO: es el efecto que tiene el taint, NoSchedule (solo se comprueba durante la planificación), NoExecution (se comprueba en tiempo de ejecución)

Taints y tolerations

Podemos indicar que un pod tolera un taint del siguiente modo:

```
---
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  tolerations:
    - key: "CPU"
      operator: "Equal"
      value: "slow"
      effect: "NoSchedule"
  containers:
    - name: dada
      image: dada:1.0 # Nombre inventado, no existe
      ports:
        - containerPort: 8080
```

Affinity/Anti-affinity sobre nodos

https:

[//kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/#affinity-and-anti-affinity](https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/#affinity-and-anti-affinity)

Por ejemplo, podemos etiquetar un nodo con una etiqueta y al ejecutar un pod indicar que tiene afinidad (`nodeAffinity`) o rechazo (`nodeAntiAffinity`) por nodos que tengan la etiqueta.

```
kubectl label nodes node1 disk:ssd
```

Las reglas a comprobar (etiquetas) se pueden especificar con `matchLabels` (para comprobar igualdad) o con `matchExpressions` (que permite más opciones). Pueden ser requeridas o bien indicar una preferencia.

Affinity/Anti-affinity sobre nodos

```
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disk
                operator: In
                values:
                  - ssd
  containers:
    - name: nginx
      image: nginx:1.29.5-alpine
      ports:
        - containerPort: 80
```

El operator puede ser: In, NotIn, Exists, o DoesNotExist.

Podemos suavizar el requisito para que sea una preferencia usando preferredDuringSchedulingIgnoredDuringExecution.

Affinity/Anti-affinity sobre pods

Necesitaremos especificar:

- Las reglas (igual que en caso de los nodos)
- El tipo de topología que usa el planificador para determinar el lugar (host `kubernetes.io/hostname`, o zona `topology.kubernetes.io/zone`) donde debe colocar el Pod.

Afinidad: el pod se debe ejecutar en el mismo lugar (host o zona) donde ya esté ejecutando otro pod.

Antiafinidad: el pod se debe ejecutar en un lugar diferente (host o zona) de donde ya está ejecutando otro pod.

Affinity/Anti-affinity sobre pods

Supongamos que dos pods que forman parte de una misma aplicación se comunican entre ellos haciendo un uso intensivo de la red. En este escenario podríamos querer que estén en el mismo nodo.

Supongamos que especificamos un pod del siguiente modo (lo etiquetamos con `layer: backend`):

Fichero: pods/pod-demo-affinity1.yaml

```
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx-backend
  namespace: ns-pods
  labels:
    layer: backend
spec:
  containers:
  - name: nginx
    image: nginx:1.29.5-alpine
    ports:
    - containerPort: 80
```

Affinity/Anti-affinity sobre pods

Al especificar el otro pod, indicamos que tiene afinidad por ejecutarse en el mismo host que el otro pod con la etiqueta layer: backend:

Fichero: pods/pod-demo-affinity2.yaml

```
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx-front
  namespace: ns-pods
  labels:
    layer: frontend
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - topologyKey: kubernetes.io/hostname
          labelSelector:
            matchExpressions:
              - key: layer
                operator: In
                values:
                  - backend
  containers:
    - name: nginx
      image: nginx:1.29.5-alpine
      ports:
        - containerPort: 80
```

Si lanzamos este y el otro pod no está ejecutándose, entonces quedará en estado *Pending*.

*Affinity/Anti-affinity sobre pods: ejemplos de escenarios**Alta disponibilidad*

Si queremos ofrecer una aplicación en alta disponibilidad no pueden caer todas las réplicas en el mismo nodo. En este caso queremos antiafinidad con el resto de pods para que las réplicas se ejecuten en nodos diferentes.

Evitar que pods que necesitan muchos recursos caigan en el mismo nodo

Supongamos una aplicación con varias réplicas que van a requerir mucho ancho de banda. En este caso también queremos antiafinidad con el resto de pods para que cada réplica se ejecute en un nodo diferente.

*Affinity/Anti-affinity sobre pods: ejemplos de escenarios**Minimizar la comunicación entre diferentes zonas de disponibilidad.*

Los proveedores cloud ofrecen diferentes zonas de disponibilidad en cada región y cobrar por la comunicación entre las zonas. Si una aplicación está formada por varios pods que se van a comunicar entre ellos, podríamos lanzar todos los pods en la misma zona de disponibilidad. En este caso buscaríamos afinidad de zona.

Tarea: desplegar un pod con mysql

- ❶ Todos los recursos se deben generar en el namespace ns-pods.
- ❷ Busca información en <https://hub.docker.com/r/mysql/mysql-server> sobre la imagen del servidor mysql y en [mysql docker](#) sobre las variables de entorno.
- ❸ Crea un fichero YAML para desplegar un pod con un contenedor a partir de la imagen mysql/mysql-server:8.0.32. Pasa variables de entorno para que el usuario root tenga la contraseña nuncahagasesto. Además, al iniciar el contenedor se debe crear la base de datos database y un usuario app-user con contraseña tampocoahagasesto con acceso a la base de datos.
- ❹ Una vez tengas el fichero YAML ejecuta el recurso en Kubernetes.
- ❺ Comprueba el estado del pod.
- ❻ Si se ha ejecutado correctamente, obtén una shell y comprueba que te puedes conectar a mysql con el usuario app-user y que se ha creado la base de datos.
- ❼ Finalmente, elimina el pod.

Índice

5 *Ejecución de cargas de trabajo*

- Pod
- Deployment
- DaemonSet
- Job y CronJob

Introducción

Los deployments se usan para aplicaciones sin estado (stateless) y proporciona una capa de abstracción adicional sobre los pods.

- Un deployment simplifica el proceso de despliegue y actualización de aplicaciones.
- Permite especificar el número de réplicas del pod que gestiona (podemos incrementar o decrementar el número).
- Permite actualizar la aplicación sin impacto para los usuarios (reemplazando las réplicas antiguas por las nuevas secuencialmente).
- Si una actualización falla, el deployment facilita volver a una versión previa de la aplicación.
- Permite mantener múltiples versiones de una aplicación en ejecución en el cluster de kubernetes.

Introducción

Una vez que las instancias han sido creadas, un controlador (específico para deployments) de Kubernetes las monitoriza.

Si un nodo que ejecuta una de las réplicas cae, el controller crea una nueva instancia en otro nodo del cluster de kubernetes.

Esto proporciona un mecanismo de *auto sanación* en caso de fallo de algún nodo o mantenimiento.

Especificación de un deployment

- `spec.replicas` permite especificar cuántas réplicas del pod se deben ejecutar
- `spec.selector` sirve para especificar a qué pod se aplica el deployment usando etiquetas.
- `spec.template` sigue el esquema de especificación de un pod pero sin especificar `apiVersion` ni `Kind`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
spec:
  replicas: 2
  selector:
    matchLabels: #El deployment selecciona pods con esta etiqueta
    app: name
  template:
    metadata:
      labels:
        app: name # Etiqueta del pod
    spec:
      containers:
        # Datos de cada contenedor en el pod
```

Ejemplo de deployment 1

Fichero:

deployments/deployment-nginx-124.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: ns-deployments
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.24-alpine
          ports:
            - containerPort: 80
```

Ejemplo de deployment II

```
kubectl apply -f deployment-nginx.yaml
```

```
kubectl get deployment -n ns-deployments  
kubectl get pod -n ns-deployments
```

Borra uno de los pods (con `kubectl delete`).

Vuelve a listar los pods del deployment y comprueba qué ocurre (el deployment ha creado otro pod ya que es lo hemos indicado en la especificación).

Cambiar el número de réplicas del deployment

Podemos incrementar el número de réplicas:

```
kubectl scale deployment nginx -n ns-deployment --replicas=3
```

```
kubectl get deployment -n ns-deployments -o wide  
kubectl get pod -n ns-deployments
```

O podemos decrementar el número de réplicas:

```
kubectl scale deployment nginx -n ns-deployments --replicas=1
```

```
kubectl get deployment -n ns-deployments -o wide  
kubectl get pod -n ns-deployments
```

Actualización de la aplicación mientras está en ejecución I

Supongamos que queremos actualizar la imagen del contenedor a nginx:1.29.5-alpine (la que hemos desplegado es nginx:1.24-alpine). Editamos el fichero yaml y actualizamos la versión:

Fichero: deployments/deployments-nginx.yaml

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: ns-deployments
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
```

Actualización de la aplicación mientras está en ejecución II

```
containers:
- name: nginx
  image: nginx:1.29.5-alpine
  ports:
  - containerPort: 80
```

Actualizamos en caliente (la aplicación se está ejecutando):

```
kubectl apply -f deployment-nginx.yaml
```

```
kubectl get deployment nginx -n ns-deployments -o wide
```

```
kubectl get pods -n ns-deployments -o wide
```

Actualización con fallo de una aplicación y vuelta al estado anterior I

Supongamos que queremos actualizar de nuevo la aplicación en el deployment pero nos equivocamos y la aplicación falla:

Fichero: deployments/deployment-nginx-error.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: ns-deployments
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.29.5-alpine
```

Actualización con fallo de una aplicación y vuelta al estado anterior II

```
command:
  - "/bin/sh"
  - "-c"
  - "sleep 20; ffmpeg"
ports:
  - containerPort: 80
```

```
kubectl apply -f deployment-nginx-error.yaml
```

```
kubectl get pods -n ns-deployments -o wide
```

Veremos cómo elimina los pods anteriores y crea los nuevos, pero al cabo de 20 segundos fallan y los vuelve a reiniciar.

En este caso podemos indicar a kubernetes que vuelva al estado previo ya que las actualizaciones se versionan:

Actualización con fallo de una aplicación y vuelta al estado anterior III

```
# Para ver las versiones
kubectl rollout history deployment nginx -n ns-deployments

# Para ir a la última versión
kubectl rollout undo deployment nginx -n ns-deployments

# Para ir a una versión concreta:
# kubectl rollout undo deployment nginx -n ns-deployments --to-revision=1
```

```
kubectl get pods -n ns-deployments -o wide
```

Tarea de deployment 1

Los recursos de esta tarea se deben desplegar en el namespace `ns-deployments`.

Escribe el yaml para un deployment con las siguientes características:

- Debe iniciar 1 réplica
- El pod tiene dos contenedores:
 - ① Contenedor con la imagen `twcam/key-value-api:1.0` (que se conecta con un servidor de pares clave/valor para almacenar y recuperar la información). Al contenedor hay que pasarle la variable de entorno `KEY_VALUE_HOST: localhost`. La aplicación escucha en el puerto 5000.
 - ② Contenedor con la imagen de `valkey/valkey:9-alpine3.23`. Esta aplicación escucha en el puerto 6379.
- Despliega el deployment
- Usa `kubect1 port-forward` para enviar las peticiones del puerto 8000 al puerto 5000 (donde escucha el contenedor del API).

Tarea de deployment II

- Comprueba que podemos realizar las siguientes peticiones:

```
curl -X POST -H 'content-type: application/json' -d  
↪ '{"name":"Eva","email":"eva@uv.es"}' http://localhost:8000/api/users/  
  
# Pon el ID que ha devuelto la petición anterior  
curl http://localhost:8000/api/users/<ID>  
  
# Pon el ID  
curl -X DELETE http://localhost:8000/api/users/<ID>
```

Índice

5 *Ejecución de cargas de trabajo*

- Pod
- Deployment
- **DaemonSet**
- Job y CronJob

Introducción

- Un **daemonset** es una abstracción que permite ejecutar un pod, y solo uno, en cada nodo del cluster.
- Por ejemplo, supongamos que queremos monitorizar los contenedores en ejecución en cada nodo del cluster. En este caso, necesitamos un pod de monitorización en cada nodo (y solo es necesario uno).
- También, es posible ejecutar el pod en un subconjunto de los nodos. Por ejemplo: supongamos que queremos monitorizar el uso de las GPUs, en este caso solo queremos desplegar el pod de monitorización en todos los nodos que tengan GPUs.
- Si un nodo cae, el pod no se recrea en otro nodo y si se añade un nuevo nodo, kubernetes automáticamente ejecutaría el pod correspondiente al daemonset.

Especificación de un daemonset

- `spec.selector` sirve para especificar a qué pod se aplica el daemonset
- `spec.template` sigue el esquema de especificación de un pod pero sin especificar `apiVersion` ni `Kind`
- `spec.template.spec.nodeSelector` permite indicar condiciones que deben cumplir los nodos para desplegar el daemonset.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: daemonset
spec:
  selector:
    matchLabels: #Selecciona el pod al que se aplica
    app: name
  template:
    metadata:
      labels:
        app: name
    spec:
      containers:
        # Datos de cada contenedor
```

*Ejemplos de posibles usos de daemonset**Monitorizar los contenedores en cada nodo*

Desplegar un daemonset que pueda obtener estadísticas de consumo de los contenedores. En este caso habría que montar en el pod el socket unix de docker, para que nuestra aplicación se comuniqué con el daemon (hay librerías en Java, Python, Go, etc para hacer esto).

Automatizar la instalación en nuevos nodos

Instalar automáticamente los drivers de nvidia, configurar el runtime, etc para que los pods puedan usar GPUs al añadir un nuevo nodo que posea GPUs (esto es una de las cosas que hace el operador de Nvidia).

Índice

5 *Ejecución de cargas de trabajo*

- Pod
- Deployment
- DaemonSet
- Job y CronJob

Introducción

Los deployments están pensados para aplicaciones que **se ejecutan indefinidamente**: representan un servicio que debe estar en ejecución para atender peticiones que pueden llegar en cualquier momento.

Sin embargo, no todas las cargas de trabajo encajan en este patrón. Por ejemplo:

- una tarea puntual que se debe ejecutar y que cuando finalice la aplicación se da por acabada (**Job**)
- una tarea que se debe ejecutar de forma periódica (**CronJob**)

Introducción

Tanto **Job** como **CronJob** se encargan de crear el pod (o los pods si se indica paralelismo) y de controlar que se cumple la especificación indicada.

Se puede especificar:

- El número de pods que se deben ejecutar en paralelo (por defecto será 1)
- El número de trabajos que deben finalizar con éxito para que el job finalice con éxito
- El número de veces máximo que se puede reiniciar si falla

Job: Declaración de Job 1

<https://kubernetes.io/docs/concepts/workloads/controllers/job/>

El siguiente ejemplo, muestra cómo se puede declarar un recurso del tipo **Job**. En este caso ejecutamos un contenedor que tiene `curl` y el trabajo consiste en descargar documentos de varias URLs.

Fichero: jobs-cronjobs/ejemplo1-job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: contador
  namespace: ns-jobs-cronjobs
spec:
  template:
    metadata:
      name: contador
      labels:
        app: contador
    spec:
      restartPolicy: Never
      containers:
```

Job: Declaración de Job II

```
- name: contador
  image: curlimages/curl:8.18.0
  command:
  - "/bin/sh"
  - "-c"
  - for u in $(seq 1 3); do curl
    ↪ "https://routerdi1315.uv.es/public/sample-docs/doc$u.pdf" -o
    ↪ /tmp/doc$u.pdf;sleep $((($RANDOM%3+1)); done
```

```
kubectl apply -f ejemplo1-job.yaml
```

```
kubectl get pods -n ns-jobs-cronjobs
```

```
pod=$(kubectl get pods -l app=contador -n ns-jobs-cronjobs -o json | jq -r
↪ .items[0].metadata.name)
kubectl logs $pod -n ns-jobs-cronjobs
```

Job: Paralelismo y número de ejecuciones exitosas

- Con `spec.parallelism` podemos indicar el número de ejecuciones en paralelo del trabajo.
- Con `spec.completions` podemos indicar el número de ejecuciones con éxito requeridas para considerar que el trabajo ha completado con éxito.

La siguiente tabla muestra las posibilidades:

Completions	Parallelism	Pods	Job finaliza
1	1	1	Cuando el proceso finaliza con 0
$N \geq 1$	$M \geq 1$	M	Cuando N pods finalizan con 0
No establecido	$M \geq 1$	M	Cuando un pod finaliza con 0 y todos los pods finalizan

Job: Ejemplo 1

En este caso lanzamos un job con 3 instancias en paralelo:

Fichero: jobs-cronjobs/job-curl-parallel.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: curl-parallel
  namespace: ns-jobs-cronjobs
spec:
  parallelism: 3
  template:
    metadata:
      name: curl-parallel
      labels:
        app: contador-concurrente
    spec:
      restartPolicy: Never
      containers:
      - name: curl-parallel
        image: curlimages/curl:8.18.0
        command:
        - "/bin/sh"
```

Job: Ejemplo II

```
- "-c"
- for u in $(seq 1 3); do curl
  ↪ "https://routerdi1315.uv.es/public/sample-docs/doc$u.pdf" -o
  ↪ /tmp/doc$u.pdf; sleep $((($RANDOM%3+1)); done
```

```
kubectl apply -f job-curl-parallel.yaml
```

```
kubectl get pods -n ns-jobs-cronjobs -l app=contador-concurrente
```

NAME	READY	STATUS	RESTARTS	AGE
curl-parallel-f6r2l	0/1	Completed	0	33s
curl-parallel-jlhw7	0/1	Completed	0	33s
curl-parallel-s76hp	0/1	Completed	0	33s

Job: Especificación del comportamiento en caso de fallo

- Con `spec.backoffLimit` se puede especificar el número de reinicios máximos en caso de que el proceso finalice con un valor distinto de 0 (el valor por defecto es 6).
- Con `spec.template.spec.restartPolicy` podemos especificar: `OnFailure` o `Never` (en este caso no tiene sentido usar `Always` ya que estamos ejecutando un servicio que recibe peticiones):
 - ▶ `OnFailure` si el proceso que se ejecuta en el contenedor falla, se reinicia el contenedor de nuevo (dentro del mismo pod).
 - ▶ `Never` si el proceso que se ejecuta en el contenedor falla, entonces se crea un nuevo pod.

Job: Ejemplo 1

En este ejemplo, hemos fijado el número máximo de reinicios a 3 y hemos indicado que debe reiniciar en caso de fallo. Forzamos el fallo ya que estamos solicitando un recurso que no existe:

Fichero: jobs-cronjobs/job-curl-error.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: curl-error
  namespace: ns-jobs-cronjobs
spec:
  backoffLimit: 3
  template:
    metadata:
      name: curl-error
      labels:
        app: job-curl-error
    spec:
      restartPolicy: OnFailure
      containers:
        - name: curl-error
```

Job: Ejemplo II

```
image: curlimages/curl:8.18.0
command: ["/bin/sh"]
args: ["-c", "curl -f
↪ https://routerdi1315.uv.es/public/sample-docs/doc100.pdf"]
```

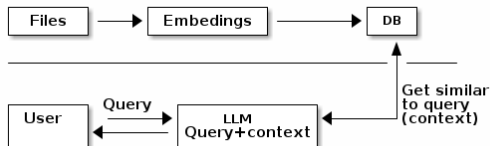
```
kubectl apply -f job-curl-error.yaml
```

```
kubectl describe job curl-error -n ns-jobs-cronjobs
```

Comprueba el comportamiento si se cambia restartPolicy a Never.

Ejemplo de Job: QA sobre documentos (LLMs) I

El QA (question/answer) sobre documentos usando LLMs funciona del siguiente modo:

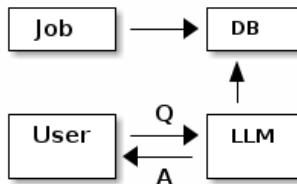


Supongamos que tenemos desplegados una bases de datos (de embeddings) y un deployment con el LLM que los usuarios pueden invocar para *conversar* con los documentos.

Ejemplo de Job: QA sobre documentos (LLMs) II

Supongamos que tenemos nuevos documentos y queremos almacenar sus embeddings en la base de datos. Este es un trabajo puntual que podríamos ejecutar en un job.

Al ejecutar el Job podríamos pasar como variable de entorno la URL del fichero zip donde están los documentos sobre los que calcular los embeddings.



Ejemplo de Job: QA sobre documentos (LLMs) III

En la implementación del contenedor que ejecutamos en el Job tendríamos que:

- Descargar los ficheros
- Obtener los tokens de cada documento
- Calcular el embedding de cada token
- Guardar los embeddings en la base de datos

Tarea: uso de job para transcodificar un vídeo

Definir un job que ejecute un contenedor con la siguiente imagen:
cloudmedialab/x264-fn-events:1.0

- La instrucción a ejecutar en el contenedor será (esta instrucción se debe pasar como segundo argumento de `/bin/sh` siendo el primer argumento `-c`)

```
ffmpeg -y -i https://routerdi1315.uv.es/video/video1.mp4 -b:v 100k  
↪ /tmp/video1-out.mp4
```

- En esta instrucción le indicamos a `ffmpeg` (que es una aplicación instalada en esa imagen) que codifique el vídeo (que tiene un bitrate de 715 kb/s) a un bitrate de 100 kb/s (por tanto a menor calidad).
- Tanto `parallelism` como `completions` deben ser 1.
- Ejecutar el job, mostrar los pods del job, y comprobar el log del pod.
- Modificar `parallelism` y `completions` para que tengan el valor 2.
- Ejecutar el job, mostrar los pods del job (ahora se deben haber ejecutado 2).

Nota: para que esto fuese funcional, tras codificar el vídeo habría que subirlo a algún servicio de almacenamiento (al finalizar la ejecución, el pod se borra y no podemos acceder).

CronJob

<https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/>

- Tal y como hemos visto, el Job se ejecuta en cuanto lo creamos.
- Sin embargo, puede haber situaciones en las que programamos una tarea para que comience a ejecutarse en un determinado momento y con una periodicidad determinada (similar a los crontab de Linux).
- Para ello se puede usar un recurso del tipo CronJob.
- Cuando llegue el momento de ejecutar el trabajo, CronJob= creará un recurso del tipo Job que ejecutará el trabajo especificado al crear el CronJob.

CronJob 1

Para especificar un CronJob:

- `spec.schedule` especifica cuándo se debe ejecutar la tarea

```
# |----- minute (0 - 59)
# | |----- hour (0 - 23)
# | | |----- day of the month (1 - 31)
# | | | |----- month (1 - 12)
# | | | | |----- day of the week (0 - 6) (Sunday to Saturday;
# | | | | | 7 is also Sunday on some systems)
# | | | | | OR sun, mon, tue, wed, thu, fri, sat
# | | | | |
# * * * * *
```

- `spec.sucessfulJobsHistory` (por defecto 3) es el número de logs se deben guardar de trabajos completados con éxito
- `spec.failedJobsHistory` (por defecto 1) es el número de logs a guardar de trabajos que han finalizado con error

CronJob II

- `spec.concurrencyPolicy` permite controlar qué hacer si el anterior cronjob no ha finalizado y ha llegado el momento de volver a ejecutar:
 - ▶ `Allow` (valor por defecto) lanza uno nuevo (sin hacer nada con el anterior)
 - ▶ `Forbid` si el anterior no ha finalizado, el nuevo no se ejecuta
 - ▶ `Replace` finaliza el anterior y ejecuta el nuevo.

CronJob: Ejemplo 1

Ejemplo de CronJob que ejecutará un Job cada minuto entre las 8:00 y las 13:00 de la mañana todos los días:

Fichero: jobs-cronjobs/cronjob.yaml

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cronjob
  namespace: ns-jobs-cronjobs
spec:
  schedule: "* 8-13 * * * "
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 5
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: busybox
              image: busybox:1.37.0
```

CronJob: Ejemplo II

```
command: ["/bin/sh", "-c", "echo Ejecutado $(date)"]
```

```
kubectl apply -f cronjob.yaml
```

```
cronjob.batch/cronjob created
```

```
kubectl get jobs -n ns-jobs-cronjobs
```

```
No resources found in ns-jobs-cronjobs namespace.
```

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 ***InitContainers***
- 7 *ConfigMap y Secret*

Índice

6

InitContainers● *InitContainers*

Introducción

Un pod puede tener uno o varios contenedores de aplicación y uno o varios contenedores de inicialización.

Los contenedores de inicialización suelen contener utilidades o scripts de instalación que no están en la imagen del contenedor de la aplicación (por ejemplo, se podría usar para iniciar las bases de datos, clonar un repositorio, etc).

Los contenedores de inicialización se ejecutan antes que los de aplicación y tienen las siguientes propiedades:

- Se ejecutan hasta su finalización y antes de los contenedores de aplicación.
- Cada contenedor de inicialización debe finalizar antes de ejecutar el siguiente. Una vez se han ejecutado, los contenedores de aplicación se pueden lanzar en paralelo.
- Si un contenedor de inicialización falla, kubelet lo reinicia hasta que tenga éxito o se alcance el número máximo de reintentos.

initContainers

<https://kubernetes.io/docs/concepts/workloads/pods/init-containers/>

Los contenedores de inicialización usan su propia imagen de contenedor (que serán diferentes de las imágenes usadas para los contenedores de la aplicación).

- Pueden usar herramientas que no estén en el contenedor de la aplicación (por ejemplo git, dig, python, curl, etc).
- Pueden tener una vista diferente del sistema de ficheros en el pod (podemos usar secrets en el contenedor de inicialización que no se usen en el de aplicación, mejora la seguridad).
- Ofrecen un mecanismo para bloquear o retrasar el inicio de los contenedores de aplicación hasta que no se cumplan una serie de precondiciones.

initContainers

Para añadir contenedores de inicialización, se añade el campo `initContainers` en la especificación del pod.

Consiste en un array (similar a los contenedores de la aplicación) donde está la especificación secuencial de contenedores.

Los contenedores de inicialización se ejecutarán secuencialmente en el orden en el que aparecen en la especificación.

initContainers 1

```
kubectl create namespace ns-initcontainers
```

Ejemplo con dos contenedores de inicialización:

Fichero: initcontainers/ejemplo1.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: initcontainers1
  namespace: ns-initcontainers
spec:
  initContainers:
    - name: wait1
      image: busybox:1.37.0
      command:
        - '/bin/sh'
        - "-c"
        - "sleep 10 && echo Wait 1 finalizado"
    - name: wait2
      image: busybox:1.37.0
      command:
```

initContainers II

```
- '/bin/sh'
- "-c"
- "sleep 10 && echo Wait 1 finalizado"
containers:
- name: nginx
  image: nginx:1.29.5-alpine
  ports:
  - containerPort: 80
```

```
kubectl apply -f ejemplo1.yaml
```

Podemos comprobar cómo evoluciona la evolución en la ejecución de los contenedores del pod si ejecutamos:

```
watch -n1 kubectl get pod initcontainers1 -n ns-initcontainers
```

initContainers III

NAME	READY	STATUS	RESTARTS	AGE
initcontainers1	0/1	Init:0/2	0	25s
NAME	READY	STATUS	RESTARTS	AGE
initcontainers1	0/1	Init:1/2	0	35s
NAME	READY	STATUS	RESTARTS	AGE
initcontainers1	1/1	Running	0	54s

initContainers

```
kubectl describe pod initcontainers1 -n ns-initcontainers
```

Para acceder a los logs de un contenedor específico dentro del pod:

```
kubectl logs initcontainers1 -c wait1 -n ns-initcontainers
```

```
kubectl logs initcontainers1 -c wait2 -n ns-initcontainers
```

Tarea sobre initContainers

- ❶ Realiza una copia del fichero `ejemplo1.yaml` al fichero `tarea-initcontainers.yaml`.
- ❷ Cambia el segundo contenedor de inicio (el que hemos llamado `wait2` por la imagen `curlimages/curl:8.18.0` donde la instrucción a ejecutar sea la descarga de <https://routerdi1315.uv.es/doc100.pdf> (que es un recurso que no existe). Pasa la opción `-f` al proceso `curl` para provocar que termine con un valor diferente de cero.
- ❸ Ejecuta el pod y comprueba qué sucede en este caso.
- ❹ Comprueba el mensaje de error que se obtiene al ejecutar `kubectl describe`.
- ❺ Borra el espacio de nombres `ns-initcontainers`.

Índice

- 1 *Introducción*
- 2 *API*
- 3 *Organización de recursos en Namespaces*
- 4 *Etiquetas y selectores*
- 5 *Ejecución de cargas de trabajo*
- 6 *InitContainers*
- 7 *ConfigMap y Secret*

Índice

- 7 *ConfigMap y Secret*
 - ConfigMap
 - Secret

Introducción

Las aplicaciones que despleguemos en kubernetes deberían estar parametrizadas para permitir su configuración mediante:

- Ficheros de configuración (**ConfigMap**)
- Variables de entorno (**ConfigMap** o **env**)

Esto permite adaptar la aplicación a diferentes entornos de ejecución sin necesidad de volver a pasar por el tedioso ciclo de modificar la aplicación, generar la imagen del contenedor, subir la imagen a un registro de imágenes, etc.

Kubernetes inyecta el contenido del ConfigMap en el contenedor que lo solicita bien como variables de entorno o como un fichero.

ConfigMap

Recurso que permite almacenar información no sensible como por ejemplo: puertos, nombre del host donde está la base de datos, URL de un API, etc

Al crear un ConfigMap con kubectl podemos proporcionar:

- Valores literales con pares clave-valor
- Un fichero que contiene pares clave-valor
- Un fichero con contenido arbitrario
- Un directorio con varios ficheros

Crear ConfigMap

Valores literales:

```
kubectl create configmap <nombre> --from-literal=port=8080  
↪ --from-literal=dbhost=mysql
```

Fichero con pares clave-valor:

```
kubectl create configmap <nombre> --from-env-file=app.env
```

Fichero con contenido arbitrario:

```
kubectl create configmap <nombre> --from-file=config.txt
```

De un directorio:

```
kubectl create configmap <nombre> --from-file=app-config-dir
```

Ejemplo de declaración de ConfigMap en fichero YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
data:
  # Formato clave-valor
  port: 8080
  # Formato fichero:
  app.properties: |
    db_host: mysql-service
    database: demo-database
```

ConfigMap de ejemplo

Fichero: configmaps/backend-config.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: ns-configmaps
data:
  db_host: mysql-service
  database: demo-database
```

```
kubectl apply -f backend-config.yaml
```

ConfigMap como variables de entorno en el pod (con el nombre asignado en el configmap) |

Fichero: configmaps/ejemplo1-uso-configmap.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ejemplo1-uso-configmap
  namespace: ns-configmaps
spec:
  containers:
    - name: busybox
      image: busybox:1.37.0
      command: ["/bin/sh"]
      args: ["-c", "sleep 3600"]
      envFrom:
        - configMapRef:
            name: backend-config
```

```
kubectl apply -f ejemplo1-uso-configmap.yaml
```

```
kubectl exec ejemplo1-uso-configmap -n ns-configmaps -- env
```

ConfigMap como variables de entorno en el pod (con el nombre asignado en el configmap) II

```
database=demo-database  
db_host=mysql-service  
# Resto de variables de entorno
```

ConfigMap como variables de entorno en el pod (cambiando el nombre de la clave) I

Fichero: configmaps/ejemplo2-uso-configmap.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ejemplo2-uso-configmap
  namespace: ns-configmaps
spec:
  containers:
    - name: busybox
      image: busybox:1.37.0
      command: ["/bin/sh"]
      args: ["-c", "sleep 3600"]
      env:
        - name: DB_HOST
          valueFrom:
            configMapKeyRef:
              name: backend-config
              key: db_host
        - name: DATABASE
          valueFrom:
            configMapKeyRef:
              name: backend-config
```

ConfigMap como variables de entorno en el pod (cambiando el nombre de la clave) II

```
key: database
```

```
kubectl apply -f ejemplo2-uso-configmap.yaml
```

```
kubectl exec ejemplo2-uso-configmap -n ns-configmaps -- env
```

```
DB_HOST=mysql-service  
DATABASE=demo-database  
# Resto de variables de entorno
```

ConfigMap como un fichero en el pod

Fichero: configmaps/backend-config-file.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config-file
  namespace: ns-configmaps
data:
  app.config: |
    db_host: mysql-service
    database: demo-database
```

```
kubectl apply -f backend-config-file.yaml
```

ConfigMap como un fichero en el pod 1

Fichero: configmaps/ejemplo3-uso-configmap.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ejemplo3-uso-configmap
  namespace: ns-configmaps
spec:
  restartPolicy: Never
  containers:
  - name: busybox
    image: busybox:1.37.0
    command: ["/bin/sh"]
    args: ["-c", "cat /app/config/app.config"]
    volumeMounts:
    - name: config
      mountPath: /app/config/app.config
      subPath: app.config
  volumes:
  - name: config
    configMap:
      name: backend-config-file
```

ConfigMap como un fichero en el pod II

Desplegamos el Pod en Kubernetes:

```
kubectl apply -f ejemplo3-uso-configmap.yaml
```

Comprobamos que el fichero se ha montado dentro del Pod en el lugar especificado (obtenemos los logs del contenedor):

```
kubectl logs ejemplo3-uso-configmap -n ns-configmaps
```

```
db_host: mysql-service  
database: demo-database
```

Tarea configmap con un fichero para nginx I

El contenedor de Nginx sirve lo que coloquemos en el directorio /usr/share/nginx/html.

Crea un configmap con el contenido del fichero index.html:

```
<html>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Uso de configmap</h1>
    Esta página ha sido creada por:
    <ul>
      <li>Nombre 1</li>
      <li>Nombre 2</li>
    </ul>
  </body>
</html>
```

Tarea configmap con un fichero para nginx II

- 1 Crea un ConfigMap en el namespace `ns-configmaps` con el contenido del fichero `index.html`.
- 2 Lanza un pod con un contenedor `nginx:1.29.5-alpine` que use este configmap y lo monte en el lugar indicado.
- 3 Comprueba que el fichero está en el pod y con el contenido del ConfigMap.
- 4 Expón el pod para que se puedan realizar peticiones desde el exterior.
- 5 Realiza una petición HTTP y comprueba que nos devuelve el fichero `index.html`.
- 6 Elimina el namespace `ns-configmaps`

Índice

- 7 *ConfigMap y Secret*
 - ConfigMap
 - Secret

Introducción

Por seguridad no debemos almacenar información crítica (como contraseñas, tokens de acceso a APIs, etc.) en los ficheros YAML de despliegue de los recursos.

Para este tipo de información Kubernetes ofrece el recurso **Secret**.

kubernetes inyecta el Secret en el contenedor que lo solicita bien como variables de entorno o como un fichero (en este sentido es similar a ConfigMap).

En el caso de los Secrets el fichero no se monta en un almacenamiento persistente (cuando el pod se elimine, también lo hará el secret).

Secrets

Se puede crear un secret usando `kubectl create secret`. A continuación hay que indicar el tipo de secret. Por ejemplo:

- **generic**: el secret se crea a partir de un fichero, un directorio o valores literales
- **docker-registry** crea un secret con las credenciales para descargar imágenes de un registro que necesita autenticación.
- **tls**: crea un secret TLS para encriptar la comunicación

Secrets a partir de literales:

Por defecto Kubernetes codifica los secrets en base 64 (lo cual no es seguro).

```
kubect1 create secret generic demo --from-literal=username=abc  
↪ --from-literal=password=123456 -n ns-secrets
```

```
kubect1 get secret demo -n ns-secrets -o=yaml
```

```
password: MTIzNDU2  
username: YWJj
```

```
# -n es para que echo no ponga un salto de línea al final del String  
echo -n "MTIzNDU2" | base64 -d
```

```
123456
```

Secrets a partir de ficheros 1

De un fichero contiene pares clave valor (con variables de entorno)

```
kubectl create secret generic db-auth --from-env-file=dbauth.env
```

De un fichero:

```
echo "Esto es una prueba" > fichero.txt  
kubectl create secret generic prueba --from-file=fichero.txt -n ns-secrets
```

```
kubectl describe secret prueba -n ns-secrets
```

```
Name:      prueba  
Namespace: ns-secrets  
Labels:    <none>  
Annotations: <none>
```

```
Type: Opaque
```

```
Data
```

```
====
```

```
fichero.txt: 19 bytes
```

Secrets a partir de ficheros II

Como vemos, ha usado como clave el nombre del fichero. Si no queremos esto, podemos especificar la clave al crear el secret:

```
kubectl create secret generic prueba --from-file=<NOMBRE-CLAVE>=fichero.txt  
↪ -n ns-secrets
```

Secrets en fichero yaml con la información ya codificada en base46

```
echo -n "abc" | base64  
echo -n "123456" | base64
```

Fichero: secrets/ejemplo1-secrets.yaml

```
apiVersion: v1  
kind: Secret  
metadata:  
  name: demo-opaque  
  namespace: ns-secrets  
type: Opaque  
data:  
  username: YWJj  
  password: MTIzNDU2
```

```
kubectl apply -f ejemplo1-secrets.yaml
```

Inyección de los secrets como variables de entorno I

Fichero: secrets/ejemplo1-uso-secrets.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: ejemplo1-busybox
  namespace: ns-secrets
spec:
  containers:
    - name: ejemplo1-busybox
      image: busybox:1.37.0
      command: ["sleep", "3600"]
      envFrom:
        - secretRef:
            name: demo-opaque
```

```
kubectl apply -f ejemplo1-uso-secrets.yaml
```

Podemos comprobar que se ha inyectado la información del secret como variables de entorno (ya decodificadas):

Inyección de los secrets como variables de entorno II

```
kubectl exec ejemplo1-busybox -n ns-secrets -- env
```

Inyección de los secrets como ficheros I

Fichero app.config:

Fichero: secrets/app.config

```
user=abc  
password=12345
```

```
kubectl create secret generic app-config --from-file=app.config -n ns-secrets
```

```
kubectl get secret app-config -n ns-secrets -o=yaml
```

Fichero: secrets/ejemplo2-uso-secrets.yaml

Inyección de los secrets como ficheros II

```
apiVersion: v1
kind: Pod
metadata:
  name: ejemplo2-busybox
  namespace: ns-secrets
spec:
  containers:
    - name: ejemplo1-busybox
      image: busybox:1.37.0
      command: ["sleep", "3600"]
      volumeMounts:
        - name: data
          mountPath: /app
          readOnly: true
  volumes:
    - name: data
      secret:
        secretName: app-config
```

```
kubectl apply -f ejemplo2-uso-secrets.yaml
```

Inyección de los secrets como ficheros III

Podemos comprobar que se ha inyectado la información del secret como un fichero (ya decodificado):

```
kubect1 exec ejemplo2-busybox -n ns-secrets -- cat /app/app.config
```

```
user=abc  
password=12345
```

Secret para TLS (HTTPS)

Supongamos que tenemos el fichero con el certificado (firmado por una CA o autofirmado) y el fichero con la clave privada.

Podemos crear un secret TLS:

```
kubectl create secret tls my-tls-secret --cert=tls.crt --key=tls.key
```

Los ficheros del secret los podemos montar en el directorio donde la aplicación lee los ficheros.

Secret para configurar el acceso a un registro de imágenes que requiere autenticación

Supongamos que vamos a desplegar en Kubernetes imágenes de contenedores que están en un registro que requiere autenticación.

Podemos crear un secret que almacene la información de autenticación, de modo que cuando un nodo necesite ejecutar la imagen pueda autenticarse con el registro.

El siguiente ejemplo muestra cómo podemos crear un secret para este propósito:

```
# Supongo que NAMESPACE y HOST_REGISTRO están establecidas
read -s -p "Docker registry user: " REG_USER
read -s -p "Docker registry password: " REG_PASSWD

kubectl create secret docker-registry my-registry-creds \
  -n ${NAMESPACE} \
  --docker-server ${HOST_REGISTRO} \
  --docker-username=${REG_USER} \
  --docker-password=${REG_PASSWD}

unset REG_USER
unset REG_PASSWD
```

Forzar a que se encripte la información de los secrets I

Para encriptar los secrets usaremos un recurso del tipo `EncryptionConfiguration`.

Necesitamos crear una clave y codificarla en base64 (a esta clave se la denomina *Customer Key*).

La clave será usada para encriptar y desencriptar la información y debe tener al menos 15 caracteres.

Una posibilidad es obtener 32 caracteres aleatorios y codificarlos en base64:

```
head -c 32 /dev/urandom | base64
```

Creamos un recurso del tipo `EncryptionConfiguration`:

Forzar a que se encripte la información de los secrets II

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources: #Recursos a encriptar
- resources:
  - secrets #Queremos encriptar secrets
  # Para ver la lista de proveedores disponibles, consultar:
  # https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/
  providers: #Lista de proveedores para encriptar
  - secretbox:
    keys:
      - name: key
        secret: "PON LA CLAVE GENERADA ANTERIORMENTE"
  - identity: {} # No encripta (por defecto)
```

Forzar a que se encripte la información de los secrets III

El fichero anterior (encryptionconf.yaml) hay que colocarlo en el directorio /etc/kubernetes/pki

A continuación hay que editar el fichero:

/etc/kubernetes/manifests/kube-apiserver.yaml y añadir en la sección command al nivel de containers la última línea mostrada en este código:

```
apiVersion: v1
kind: Pod
spec:
  containers:
    - command:
      - kube-apiserver
      - --encryption-provider-config=/etc/kubernetes/pki/encryptionconf.yaml
```

Tras cambiar la configuración se reinicia kube-apiserver y si todo ha ido bien (esto no lo he probado), los secrets (creados a partir de este momento) se guardarán encriptados en etcd.

Tarea secrets y configmap para configurar Nginx I

- 1 Todos los recursos se crearán en el namespace `ns-secrets`
- 2 Generar una clave y un certificado autofirmado (adaptado de este [enlace](#)):

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout nginx.key  
↪ -out nginx.crt
```

- 3 Crear un secret para cada uno de los ficheros generados
- 4 Crear un configmap con el contenido del siguiente fichero de configuración de Nginx:

Fichero: `secrets/nginx.conf`

Tarea secrets y configmap para configurar Nginx II

```
user  nginx;
worker_processes  2;
events {
    use              epoll;
    worker_connections  128;
}
http {
    index index.html;
    server {
        listen 443 ssl;
        ssl_certificate      /etc/nginx/cert/tls.crt;
        ssl_certificate_key  /etc/nginx/cert/tls.key;
    }
}
```

- ⑤ Desplegar un pod con Nginx y montar el configmap (en /etc/nginx/nginx.conf) y los dos secrets en las rutas configuradas en el fichero de configuración.
- ⑥ Redirigir las peticiones al puerto 8443 de la máquina local al puerto 443 de nginx (con `kubectl port-forward`).

Tarea secrets y configmap para configurar Nginx III

- 7 Abrir en el navegador: <https://localhost:8443> y ver si es correcto.